ESCUELA DE INGENIERÍA ELECTROMECÁNICA

ÁREA ELÉCTRICA

Reporte: Laboratorio II
Programación de funciones combinacionales

Elizabeth Bolaños Vargas	2022437703
Guillermo Obando Solís	2021117231
Gerardo Bonilla Soto	2022111457

18 de marzo de 2026

Índice general

1. Objetivos: 2
2. Resultados 3
   2.1. Actividad 1 3
   2.2. Actividad 2 7
3. Análisis de resultados 10
4. Conclusiones 16

1. Objetivos:

- Programar funciones booleanas combinacionales en Arduino.
- Verificar la compuertas NAND y NOR son en si mismas un conjunto Universal, equivalentes a el conjunto de conectivas lógicas OR, AND y NOT.
- Verificar que a partir de los mintérminos de una expresión lógica se obtienen los circuitos con compuertas NAND y de los maxtérminos, los circuitos con compuertas NOR.
- Verificar que para obtener la solución mínima de un circuito lógico, es necesario sintetizar los circuitos tanto por mintérminos como por maxtérminos.
- Programar multiplexores 4×1 y 8×1.

Materiales:

- Arduino UNO.
- 10resistencias de 330 Ω.
- 5 resistencias de 1K Ω.
- 10 interruptores pulsadores.
- 1 protoboard.
- 10 diodos leds.
- Jumpers o cables.

2. Resultados

2.1. Actividad 1

Procedimiento:

Se tiene que una planta industrial se desea automatizar un motor y una alarma de acuerdo a 4 sensores llamados a, b, c y d.

La señal del motor se representa por la función lógica f1, y se activará cuando los sensores a y b estén encendidos, o cuando los sensores b y d estén encendidos, o cuando los sensores a, c y d están encendidos. Lo planteado anteriormente se puede representar mediante la siguiente función:

$$f_1 = ab \ + bd \ + acd \tag{1}$$

Explicación del código:

```
/*Se establecen los vectores con las entradas y las salidas
ademas de utilizar boolean para definir los false de una manera mas
   practica
evitando el estar definiendo tantas variables, se usan vectores para
   acortar*/
const int PinEntrada[4] = {2,3,4,5};
const int PinSalf1[3] = {6,7,8};
const int PinSalf2[3] = {9,10,11};
boolean entradas[4]; //mismo numero que las entradas
boolean resultlog[6] = {false,false,false,false,false,false};

/*En void setup se utiliza el ciclo for para poder leer las entradas
mediante la suma de i++ se logra establecer las 4 entradas de los pines
recordar que los valores van de 0 a 3
Se hace lo mismo para establecer los pines de salida*/
  void setup() {
    // put your setup code here, to run once:
    Serial.begin(9600);


//Se establecen los pines de entrada
    for(int i = 0; i < 4; i++) {
      pinMode(PinEntrada[i], INPUT);
    }

//Se establecen los pines de salida de la funcion 1
    for(int i = 0; i < 3; i++ ) {
      pinMode(PinSalf1[i], OUTPUT);
    }

//Se establecen los pines de salida de la funcion 2
    for(int i = 0; i < 3; i++){
      pinMode(PinSalf2[i], OUTPUT);
```

```
    }
  }

/*En void loop se realizan la lectura de las entradas y se escriben las
   salidas*/
  void loop() {
    // put your main code here, to run repeatedly:

    //se leen las entradas
    for(int i=0; i<4; i++){
      entradas[i] = digitalRead(PinEntrada[i]);
    }

    //programa:
    //en este se establece que  va a tener cada una de las funciones como
       entradas , y esto se guarda dentro de la lista resultlog

    resultlog[0] = f1SDP(entradas[0],entradas[1],entradas[2],entradas[3]);
    resultlog[1] = f1PDS(entradas[0],entradas[1],entradas[2],entradas[3]);
    resultlog[2] =f1NAND(entradas[0],entradas[1],entradas[2],entradas[3]);
    resultlog[3] = f2SDP(entradas[0],entradas[1],entradas[2],entradas[3]);
    resultlog[4] = f2PDS(entradas[0],entradas[1],entradas[2],entradas[3]);
    resultlog[5] = f2NOR(entradas[0],entradas[1],entradas[2],entradas[3]);



    //se escriben las salidas
    for(int i = 0; i < 3; i++) {
      digitalWrite(PinSalf1[i],resultlog[i]);
    }

    for(int i = 0; i < 3; i++) {
      digitalWrite(PinSalf2[i],resultlog[i+3]);
    }

    //Imprimir entradas:
    Serial.print("Entradas:");
    for(int i = 0; i < 4; i++){
      Serial.print(entradas[i]);
      Serial.print(" ");
    }
    //Imprimir salidas:
    Serial.print("Salidas F1:");
    for(int i =0; i < 3; i++){
      Serial.print(resultlog[i]);
      Serial.print(" ");
    }

    Serial.print("Salidas F2:");
```

```
    for(int i =3; i < 6; i++){
      Serial.print(resultlog[i]);
      Serial.print("␣");
    }
    Serial.println();

    delay(200);
  }
//F1
  bool f1SDP(bool a,bool b,bool c,bool d){
    return (a && b) || (b && d) || (a && c && d);
  }


  bool f1PDS(bool a, bool b, bool c, bool d){
    return (a || b) && (a || d) && (b || d) && (b || c);
  }

  bool f1NAND(bool a, bool b, bool c, bool d){
    return !(!(a && b) && !(b && d) && !(a && c && d));
  }


//F2
  bool f2SDP(bool a, bool b, bool c, bool d){
    return (a && !b && !c && !d) || (!a &&  !b && c) || (a && b && c) || (
       a && b && d);
  }

  bool f2PDS(bool a, bool b, bool c, bool d){
    return (a || c) && ( a || !b) && (b || c || !d) && (!b || c || d) &&
       (!a || b || !c);
  }

  bool f2NOR(bool a, bool b, bool c, bool d){
    return !(!(a || c) || !(a || !b) || !(b || c || !d) || !(!b || c || d)
        || !(!a || b || !c));
  }
```

Tablas de la verdad:

Tabla 1: Tabla de verdad de f_1.

a	b	c	d	$f_1(a,b,c,d)$
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

De acuerdo con la función F1 el programa debe ser capaz de realizar las siguientes operaciones,la suma de productos (SPD), el producto de las sumas (PDS) y la función resuelta por NAND/NAND:

$$f_{1SDP} = ab + bd + acd \tag{2}$$

$$f_{1PDS} = (a+b)\cdot(a+d)\cdot(b+d)\cdot(b+c) \tag{3}$$

$$f_{1NAND/NAND} = \overline{\overline{ab}\cdot\overline{bd}\cdot\overline{acd}} \tag{4}$$

Para la siguiente parte se tiene que la señal de alarma se representa con la función lógica f2, y se activará siempre que estén todos los sensores apagados excepto a o todos apagados excepto c o estén encendidos c y d y el resto apagados. También la alarma se enciende cuando todos los sensores están encendidos o cuando están todos encendidos excepto d o todos encendidos excepto c.

Lo anteriormente planteado se puede ver en la siguiente ecuación:

$$f2 = a\bar{b}\bar{c}\bar{d} \ + \bar{a}\bar{b}c\bar{d} \ + \bar{a}\bar{b}cd \ + abcd \ + abc\bar{d} \ + ab\bar{c}d \tag{5}$$

Que al simplificarlo se puede ver tipo:

$$f2 = a\bar{b}\bar{c}\bar{d} \ + \bar{a}\bar{b}c \ + abc \ + abd \tag{6}$$

Tabla 2: Tabla de verdad de f_2.

a	b	c	d	$f_2(a,b,c,d)$
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Se tiene que se debe de obtener que el programa realice tres funciones, siendo la suma de productos (SPD), el producto de las sumas (PDS) y la función resuelta por NOR/NOR:

$$f_{2SDP} = a\bar{b}\bar{c}\bar{d} + \bar{a}\bar{b}c + abc + abd \tag{7}$$

$$f_{2PDS} = (a+c)(a+\bar{b})(b+c+\bar{d})(\bar{b}+c+d)(\bar{a}+b+\bar{c}) \tag{8}$$

$$f_{2NOR/NOR} = \overline{\overline{(a+c)} + \overline{(a+\bar{b})} + \overline{(b+c+\bar{d})} + \overline{(\bar{b}+c+d)} + \overline{(\bar{a}+b+\bar{c})}} \tag{9}$$

2.2. Actividad 2

Procedimiento:

Se debe de programar para este caso que el arduino funcione como:

- Un multiplexor 2x1.
- Un multiplexor 4x1.
- Un multiplexor 8x1.

El multiplexor 8x1 debe de tener **a** como variable ingresada y obtener el comportamineto de la Tabla 2.2.

Por otro lado, se hace dos funciones para que funcionen como el mutiplexor 4x1 y una que sea 2x1, el cual es el que va a unir las salidas de los Mux 4x1 en una única salida. Y de esa manera comportarse como un Mux 8x1 ya que tendría 8 entradas y 1 única salida.

Explicación del código:

```
//Se establece la lista boleana de las entradas, las cuales todas son 0
boolean result[4]; //en esta lista se guardaran los resultados de las funciones de los MUX
boolean entr[4] = {false,false,false,false};
//Se establece la lista de los pines input y output
const int In[4] = {2,3,4,8};
const int Out[2] = {6,7};

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  //Se configuran los pines de entrada y los de salida mediante el ciclo for:

  for(int i = 0; i<4; i++){
    pinMode(In[i],INPUT);
  }

  for(int i = 0; i<2; i++){
    pinMode(Out[i],OUTPUT);
  }

}

void loop() {
  // put your main code here, to run repeatedly:
  //Se lee lo que hay en el pin de entrada:
  for(int i=0; i<4; i++){
   entr[i] = digitalRead(In[i]);
  }

  //Se establecen que van en cada una de las funciones y en que posicion de la lista ira:
  result[0] = MUX41(entr[0],true,true,!entr[0],entr[2],entr[3]);
  result[1] = MUX42(!entr[0],entr[0],entr[0],false,entr[2],entr[3]);
  result[2] = MUX2X1(result[0],result[1],entr[1]);
  result[3] = MUX8X1(entr[0],true,true,!entr[0],!entr[0],entr[0],entr[0],false,entr[3],entr[2],entr[1]);

  //Se escriben las salidas:
  digitalWrite(Out[0],result[3]);
  digitalWrite(Out[1],result[2]);

  //Se imprimen las entradas:
  Serial.print("a␣=␣");
  Serial.println(entr[0]);
  Serial.print("b␣=␣");
```

```
  Serial.println(entr[1]);
  Serial.print("c␣=␣");
  Serial.println(entr[2]);
  Serial.print("d␣=␣");
  Serial.println(entr[3]);

  //Se imprimen las salidas:
  Serial.print("Salida␣Mux8:␣");
  Serial.println(result[3]);
  Serial.print("Salida␣Mux2:␣");
  Serial.println (result[2]);

  delay(400);
}

//Funciones4x1
bool MUX41 (bool I0, bool I1, bool I2, bool I3, bool S0, bool S1) {
  return (I0 && !S1 && !S0) || (I1 && !S1 && S0) || (I2 && S1 && !S0) || (
      I3 && S1 && S0);
}
bool MUX42 (bool I4, bool I5, bool I6, bool I7, bool S0, bool S1) {
  return (I4 && !S1 && !S0) || (I5 && !S1 && S0) || (I6 && S1 && !S0) || (
      I7 && S1 && S0);
}

//Funcion 2x1
bool MUX2X1(bool MUX41, bool MUX42, bool S0) {
  return (MUX41 && !S0) || (MUX42 && S0);
  }

   //Funcion del 8X1:
bool MUX8X1 (bool I0,bool I1, bool I2, bool I3, bool I4,bool I5, bool I6,
    bool I7, bool S0, bool S1, bool S2) {
  return  (I0 && !S0 && !S1  && !S2)||
          (I1 &&  S0 && !S1  && !S2)||
          (I2 && !S0 &&  S1  && !S2)||
          (I3 &&  S0 &&  S1  && !S2)||
          (I4 && !S0 && !S1  && S2 )||
          (I5 &&  S0 && !S1  && S2 )||
          (I6 && !S0 &&  S1  && S2 )||
          (I7 &&  S0 &&  S1  && S2 );
}
```

Tabla 3: Comportamiento esperado de la estructura lógica.

a	b	$F(a,b)$
0	0	$\overline{c+d}$
0	1	$c+d$
1	0	$\overline{c \cdot d}$
1	1	$c \oplus d$

Tabla de la verdad:

Tabla 4: Tabla de verdad de la actividad 2.

a	b	c	d	$Salida$
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

3. Análisis de resultados

En el desarrollo de esta sección se encuentra que en la Actividad 1 se implementaron las funciones lógicas f_1 y f_2 mediante programación en Arduino utilizando diferentes representaciones de las funciones booleanas: *suma de productos* (SDP), *producto de sumas* (PDS) y utilizando compuertas universales NAND y NOR.

A partir de las pruebas realizadas con el circuito físico y la lectura de las entradas provenientes de los pulsadores, se construyeron las tablas de verdad experimentales para ambas funciones. Al comparar dichas tablas con las tablas de verdad teóricas presentadas anteriormente, se observó que todas las combinaciones de entrada producen exactamente las mismas salidas esperadas. Esto confirma que la implementación programada reproduce correctamente el comportamiento lógico definido por las ecuaciones booleanas.

Además, los resultados permiten verificar que las compuertas universales NAND y NOR pueden uti-

lizarse para implementar las mismas funciones lógicas que las combinaciones de compuertas AND, OR y NOT, tal como se establece en la teoría del álgebra booleana. La equivalencia entre las formas SDP, PDS y las implementaciones con NAND/NAND o NOR/NOR se comprobó experimentalmente, ya que todas produjeron salidas idénticas para cada combinación de entradas. Esto demuestra que diferentes representaciones algebraicas de una función lógica pueden conducir al mismo comportamiento en un circuito digital.

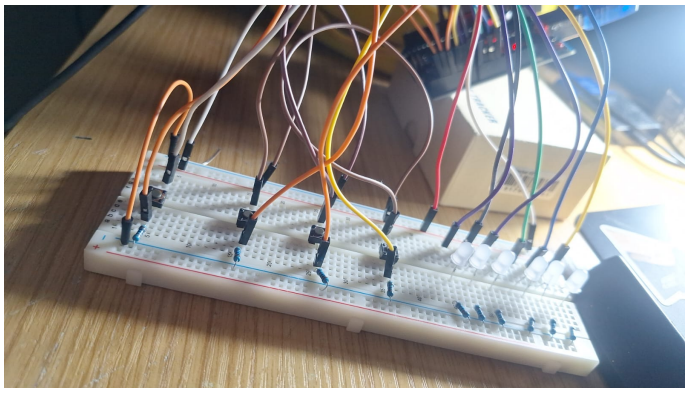

Figura 1: Montaje del circuito implementado para la Actividad 1 en protoboard

En la Actividad 2 se implementó un sistema lógico combinacional utilizando multiplexores con el objetivo de reproducir el comportamiento definido en la tabla de verdad teórica correspondiente. Para ello, se desarrollaron dos enfoques equivalentes: un multiplexor 8×1 y una configuración compuesta por dos multiplexores 4×1 junto con un multiplexor 2×1, los cuales fueron programados en Arduino y evaluados experimentalmente.

A partir de los resultados obtenidos, representados en la Tabla 4, se observa que las salidas generadas coinciden en todas las combinaciones con el comportamiento esperado definido en la tabla teórica. Específicamente, para cada combinación de las variables de entrada a y b, se obtienen correctamente las operaciones lógicas asociadas ($c + d$, $c \cdot d$ y $c \oplus d$), lo cual evidencia que la implementación del sistema reproduce de manera adecuada la lógica planteada.

Asimismo, se observa que ambas configuraciones de multiplexores (MUX 8×1 y la combinación de MUX 4×1 con MUX 2×1) presentan el mismo comportamiento para todas las combinaciones de entrada. Esto indica que, a pesar de emplear estructuras diferentes, es posible obtener una misma función lógica, lo que confirma la equivalencia funcional entre ambas implementaciones.

Desde el punto de vista de la programación, el código desarrollado permitió estructurar adecuadamente las entradas, definir funciones específicas para cada tipo de multiplexor y comparar simultáneamente las salidas de ambas configuraciones mediante el uso de pines independientes. Esto facilitó la verificación experimental directa, al evidenciar que no se presentan discrepancias entre las salidas obtenidas.

Adicionalmente, se evidencia que la expansión de la tabla teórica a una tabla completa de todas las combinaciones de las variables a, b, c y d permite validar el comportamiento del sistema de forma

integral, asegurando que la lógica implementada se mantiene consistente en todos los casos posibles.

Como complemento al análisis anterior, en las siguientes imágenes se presenta la evidencia experimental del montaje y funcionamiento del sistema, mediante la cual se respalda el comportamiento del multiplexor observado en la tabla de verdad práctica, mostrando el funcionamiento de las compuertas.

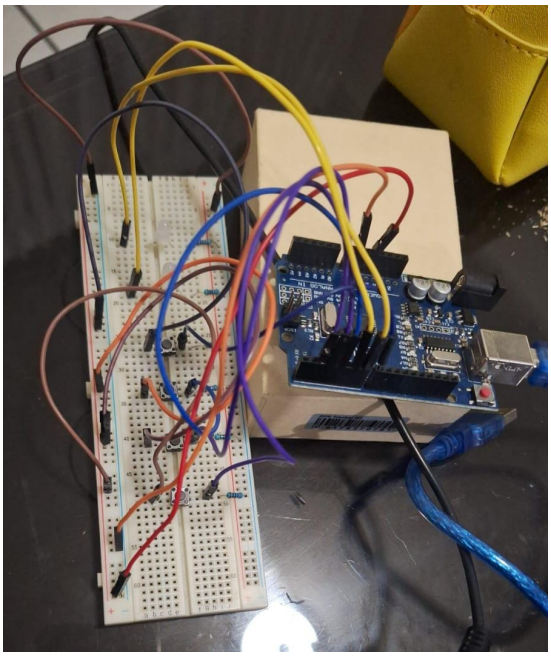

Figura 2: Montaje del circuito implementado para la Actividad 2 en protoboard

Evidencia experimental

Actividad 1

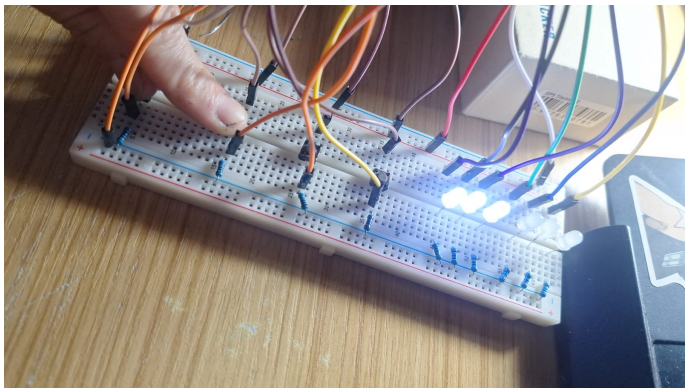

Figura 3: Demostración experimental del funcionamiento del sistema mediante las salidas obtenidas

```
Message (Enter to send message to 'Arduino UNO' on 'COM3')

Entradas:1 0 0 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:1 0 0 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:1 0 0 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:1 0 0 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:1 0 0 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:1 0 0 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:1 0 0 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:1 0 0 0 Salidas F1:0 0 0 Salidas F2:1 1 1
```

Figura 4: Evaluación de salidas de funciones lógicas F1 y F2 para entradas constantes en Arduino

```
Entradas:1 1 1 0 Salidas F1:1 1 1 Salidas F2:1 1 1
Entradas:1 0 1 0 Salidas F1:0 0 0 Salidas F2:0 0 0
Entradas:1 0 1 0 Salidas F1:0 0 0 Salidas F2:0 0 0
Entradas:1 1 1 0 Salidas F1:1 1 1 Salidas F2:1 1 1
Entradas:1 1 1 0 Salidas F1:1 1 1 Salidas F2:1 1 1
Entradas:1 1 1 0 Salidas F1:1 1 1 Salidas F2:1 1 1
Entradas:1 1 1 0 Salidas F1:1 1 1 Salidas F2:1 1 1
Entradas:1 1 1 0 Salidas F1:1 1 1 Salidas F2:1 1 1
```

Figura 5: Evaluación de salidas de funciones lógicas F1 y F2 para entradas constantes en Arduino

```
Entradas:0 0 1 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:0 0 1 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:0 0 1 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:0 0 1 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:0 0 1 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:0 0 1 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:0 0 1 0 Salidas F1:0 0 0 Salidas F2:1 1 1
Entradas:0 0 1 0 Salidas F1:0 0 0 Salidas F2:1 1 1
```

Figura 6: Evaluación de salidas de funciones lógicas F1 y F2 para entradas constantes en Arduino

```
Output   Serial Monitor X
Message (Enter to send message to 'Arduino UNO' on 'COM3')

Entradas:0 1 0 1 Salidas F1:1 1 1 Salidas F2:0 0 0
Entradas:0 1 0 1 Salidas F1:1 1 1 Salidas F2:0 0 0
Entradas:0 1 0 1 Salidas F1:1 1 1 Salidas F2:0 0 0
Entradas:0 1 0 1 Salidas F1:1 1 1 Salidas F2:0 0 0
Entradas:0 1 0 1 Salidas F1:1 1 1 Salidas F2:0 0 0
Entradas:0 1 0 1 Salidas F1:1 1 1 Salidas F2:0 0 0
Entradas:0 1 0 1 Salidas F1:1 1 1 Salidas F2:0 0 0
Entradas:0 1 0 1 Salidas F1:1 1 1 Salidas F2:0 0 0
```

Figura 7: Evaluación de salidas de funciones lógicas F1 y F2 para entradas constantes en Arduino

Actividad 2

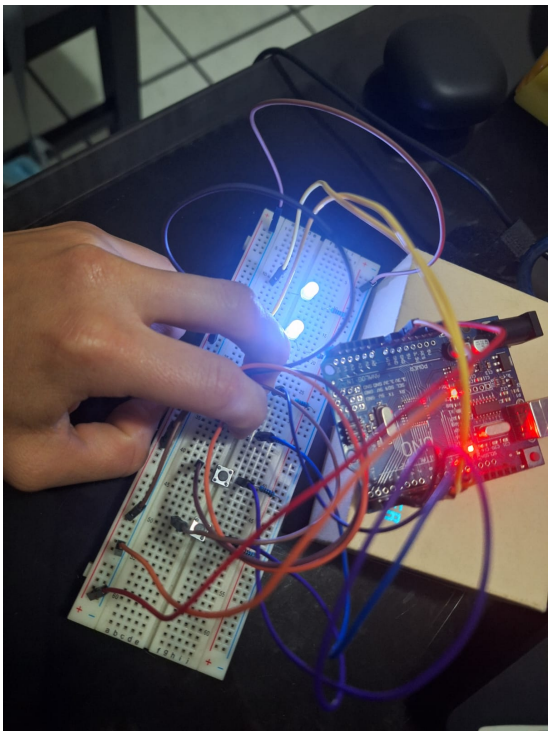

Figura 8: Demostración experimental del funcionamiento del sistema mediante las salidas obtenidas

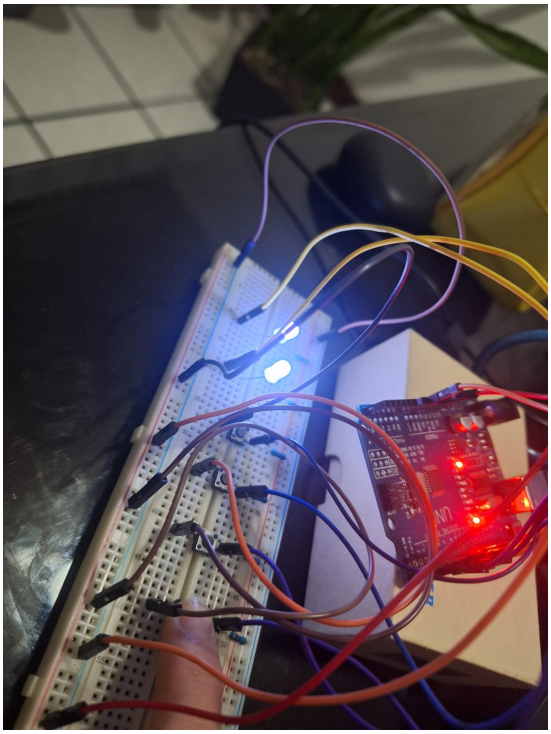

Figura 9: Demostración experimental del funcionamiento del sistema mediante las salidas obtenidas

```
Salida Mux8: 0
Salida Mux2: 0
a = 0
b = 0
c = 0
d = 0
Salida Mux8: 0
Salida Mux2: 0
a = 0
b = 0
c = 0
d = 0
Salida Mux8: 0
Salida Mux2: 0
a = 0
b = 0
c = 0
d = 0
```

Figura 10: Verificación de salidas en multiplexores (MUX 8:1 y 2:1) para una combinación de entradas específica

```
a = 1
b = 1
c = 0
d = 1
Salida Mux8: 1
Salida Mux2: 1
a = 1
b = 1
c = 0
d = 1
Salida Mux8: 1
Salida Mux2: 1
```

Figura 11: Verificación de salidas en multiplexores (MUX 8:1 y 2:1) para una combinación de entradas específica

```
a = 1
b = 1
c = 0
d = 0
Salida Mux8: 0
Salida Mux2: 0
```

Figura 12: Verificación de salidas en multiplexores (MUX 8:1 y 2:1) para una combinación de entradas específica

```
Salida Mux8: 1
Salida Mux2: 1
a = 0
b = 0
c = 1
d = 1
Salida Mux8: 1
Salida Mux2: 1
a = 0
b = 0
c = 1
d = 1
Salida Mux8: 1
Salida Mux2: 1
```

Figura 13: Verificación de salidas en multiplexores (MUX 8:1 y 2:1) para una combinación de entradas específica

4. Conclusiones

- Se logró programar correctamente funciones booleanas combinacionales en Arduino, verificando mediante las pruebas experimentales que las salidas obtenidas coinciden con las tablas de verdad teóricas para todas las combinaciones de entrada.
- Se comprobó que las compuertas NAND y NOR constituyen un conjunto universal de compuertas lógicas, ya que fue posible implementar las funciones booleanas planteadas utilizando únicamente estas compuertas y obtener el mismo comportamiento que con las compuertas AND, OR y NOT.
- Se verificó que para obtener una solución lógica mínima es necesario analizar la función tanto por mintérminos como por maxtérminos. En la Actividad 1 se compararon las formas SDP, PDS y sus equivalentes con compuertas universales, comprobándose que todas producen el mismo comportamiento lógico esperado.
- Se logró programar correctamente multiplexores 4×1 y 8×1 en Arduino, comprobando en la Actividad 2 que tanto el MUX 8×1 como la combinación de dos MUX 4×1 con un MUX 2×1 reproducen el comportamiento definido en la tabla de verdad teórica. La coincidencia entre la tabla experimental y la teórica evidenció que los multiplexores no solo funcionan como selectores de datos, sino también como una herramienta válida para implementar funciones lógicas combinacionales.